Fair four-controller comparison: what it does and why it is correct

July 2026

Summary. Four controllers, one shared constraint polytope, same plant, same stressed day. Worst-consumer demand met: KPC 97.561 % (96/96 certified-optimal QP solves, 992 ms median); iterated Koopman-LMPC 97.735 % (93/96); exact-model NMPC 94.582 % (first-order optimal on 29/96 steps, about 81× KPC's solve time); Jacobian-LMPC 91.623 %. Every number is read from the shipped `.mat` files. An independent rerun reproduces the convex rows exactly (Section 1).

1 What this component does and what I did

Four controllers run the same stressed 24-hour day (`mdot_scale` = 0.35, i.e. 35 % of nominal flow; horizon $N_p = 12$; 96 control steps). They differ only in prediction model and optimiser:

controller	prediction model	optimiser
KPC (this work)	direct multi-step Koopman lift, one map per horizon (no rollout)	convex QP
iterated Koopman-LMPC	same 47-feature lift, one step at a time	convex QP
Jacobian-LMPC	finite-difference linearisation of the exact plant	convex QP
NMPC	the exact nonlinear plant	`fmincon` (SQP)

Why the fight is fair.

- *The baselines are oracles:* NMPC and Jacobian-LMPC predict by re-integrating the exact plant from its exact internal state (`air_roll.m`), while KPC and the iterated baseline see only the learned lift. The handicap runs against KPC, not for it.
- *Identical actuation:* the actuated degrees of freedom are the same for all four; KPC's soft slacks and its internal $T^{(i,r)}$ variable are predictor mechanisms, never applied to the plant.
- *One shared constraint polytope* (mirrors KPC's constants): flow box $[0.5694\,m_d,\ 1.5\,m_d]$, rate caps $|\Delta T^{(0,s)}| \le 7.5\,\mathrm{K}$ and $|\Delta r_q| \le 4.5\,\mathrm{kg/s}$, supply box $T^{(0,s)} \in [17.69, 26]\,^\circ\mathrm{C}$. Mass conservation holds by construction via the null-space $r_q = m_d + Nx$, $N = \mathrm{null}([\,M_{\mathrm{supply}};\ M_{\mathrm{return}}\,])$; the shared adequacy floor equals KPC's per-horizon floor under $N_c = 1$; non-emptiness is checked with `linprog`.

Two cost conventions.

- *Primary* (`ls`): the baselines minimise a neutral squared-deficit cost $J_{\mathrm{ls}} = \sum_{i,h}((c_{ih} - d_{ih})/10^3)^2$. KPC winning under its own asymmetric cost against this neutral one is conservative.

- *Sensitivity* (`kpc`): the baselines instead get KPC's exact production objective on their exact-plant rollout (`air_kpc_cost.m`),

$$J_{\text{kpc}} = -\sum_{i,h} c_{ih} + \alpha \sum_{h} \left(T^{(0,s)} - T_h^{(0,r)}\right) Q_{\text{net},h} + R_{\Delta T} \left(\Delta T^{(0,s)}\right)^2 + R_{\Delta q} \left\|\Delta r_q\right\|^2. \quad (1)$$

Result. KPC ties the iterated baseline on worst-consumer demand met, and both beat NMPC and Jacobian-LMPC. The reliability gap matters as much as the ranking: KPC is feasible and optimal on 96/96 steps at 992 ms median, while NMPC is first-order optimal on only 29/96 (66 stall at `StepTolerance`, 1 hard failure) at about 81× the solve time, without exhausting its budget (max 4 of 1000 iterations).

controller	prediction model	worst met [%]	solver status	median solve [ms]
KPC (direct, convex QP)	learned lift	97.561	96/96 optimal	992
Koopman-LMPC (iterated, QP)	learned lift	97.735	93/96 optimal	656
NMPC (`fmincon` SQP)	exact plant	94.582	29/96 first-order	80 346
Jacobian-LMPC (convex QP)	exact plant	91.623	96/96 optimal	17 658

- *Wider-box (part-4) setup* (`visualizations/4_comparison`): NMPC lands at 95.5% and Jacobian-LMPC at 87.3% on the same day; the difference is exactly the equalisation items above, the Koopman rows unchanged.
- *Nonconvexity* (`kpc` mode): the convex legs hold, the nonconvex one does not. Jacobian-LMPC drops to 90.019%; NMPC collapses to 44.909% (66/96 hard failures, no step first-order optimal) because the exact-plant rollout diverges at points `fmincon` probes. Failures degrade gracefully: a 10^6 guard replaces non-finite objectives, a failed step holds u_{prev} (the same fallback KPC's QP uses), and cap hits are printed, never hidden.
- *Efficiency:* the comfort tie is not bought with energy. At essentially equal delivered heat (523.41 vs 523.48 kWh) KPC draws 932.86 kWh source energy against the iterated baseline's 981.28 kWh, so KPC is 4.9% more efficient (intensity $E_{\text{src}}/E_{\text{del}}$ 1.7823 vs 1.8745), the source energy computed by the same formula for all four. The comparison is deliberately between the two controllers that meet comfort. The other two show a lower intensity (Jacobian-LMPC 1.2564, NMPC 1.6073), but they buy it by running the source cooler and missing demand (91.6 % and 94.6 % worst-consumer), so on this plant intensity only says something once comfort is met.
- *Shared tuning, honest negative:* sharing KPC's tuned $N_p = 12$ does not disadvantage the baselines (the iterated one is flat across the horizon, the Jacobian gains only 0.15 pp at $N_p = 16$); KPC is the horizon-sensitive one, and at $N_p = 8$ the iterated baseline beats it.

worst met [%]	$N_p = 8$	$N_p = 12$	$N_p = 16$
KPC	89.409	97.561	97.676
Koopman-LMPC	97.670	97.735	97.735
Jacobian-LMPC	88.264	91.623	91.775

- *FD step:* the one numerically chosen baseline parameter, the Jacobian-LMPC's finite-difference step, favours the baseline (`fd_T0s`= 2.0 gives 91.623% against 88.376% at 0.5).
- *Reproducibility:* a second full run with a tighter NMPC budget (`StepTolerance` 10^{-6}, 50 iterations, 500 evaluations) reproduces the three deterministic legs to the third decimal (97.561 / 97.735 / 91.623); only the `fmincon` leg moves (94.139 vs 94.582, 0.44 pp; positive exitflags on 92/96 against 95/96). The QP rows are exactly reproducible, the NMPC row stable to about half a percentage point, stated as measured.

Code layout

file	role
`run_comparison_airtight.m`	the main run: four controllers, one setup, saves the `.mat`
`air_build_cons.m`	the one shared constraint polytope (mirrors KPC's constants)
`air_roll.m`	exact-plant rollout used by both plant-model baselines
`air_nmpc_solve.m`, `air_jlmpc_solve.m`	the two baseline solvers
`air_kpc_cost.m`	KPC's production objective for the sensitivity mode
`sanity_airtight.m`	the setup-fidelity gate (must pass before any re-run)
`sweep_cheap.m`, `viz_airtight.m`	the fairness sweeps and the three figures
`airtight_comparison_ls.mat`	the headline run (the table above)
`airtight_comparison_kpc.mat`	the equalised-cost sensitivity run (the nonconvexity bullet)
`airtight_comparison_ls_rerun.mat`	the independent rerun at a tighter NMPC budget
`sweep_cheap_ls.mat`	the horizon and finite-difference sweeps

2 How to reproduce

All commands run from the repository root (see README). The reported numbers need no run: load any shipped `.mat` file and inspect struct `M`. The gate `sanity_airtight.m` must pass first (it re-runs the two Koopman legs and asserts they land within 0.05 pp of 97.561 and 97.735). The headline run is `matlab -batch "AIR_COSTMODE='ls'; AIR_TAG='ls'; run('visualizations/fair_comparison/run_comparison_airtight.m')"` (about three hours, NMPC-dominated); the same with `AIR_COSTMODE='kpc'` gives the cost-sensitivity run. `viz_airtight.m` renders the three figures from the saved data in seconds. `sweep_cheap.m` regenerates the horizon and finite-difference sweeps, and despite its name it is not a quick script: it re-runs the closed loop once per setting, so it takes on the order of an hour.

3 The figures

All three figures render from the shipped `.mat` files via `viz_airtight.m`, so they show the exact run behind the table.

3.1 Closed-loop tracking and solve times (`viz_airtight.pdf`)

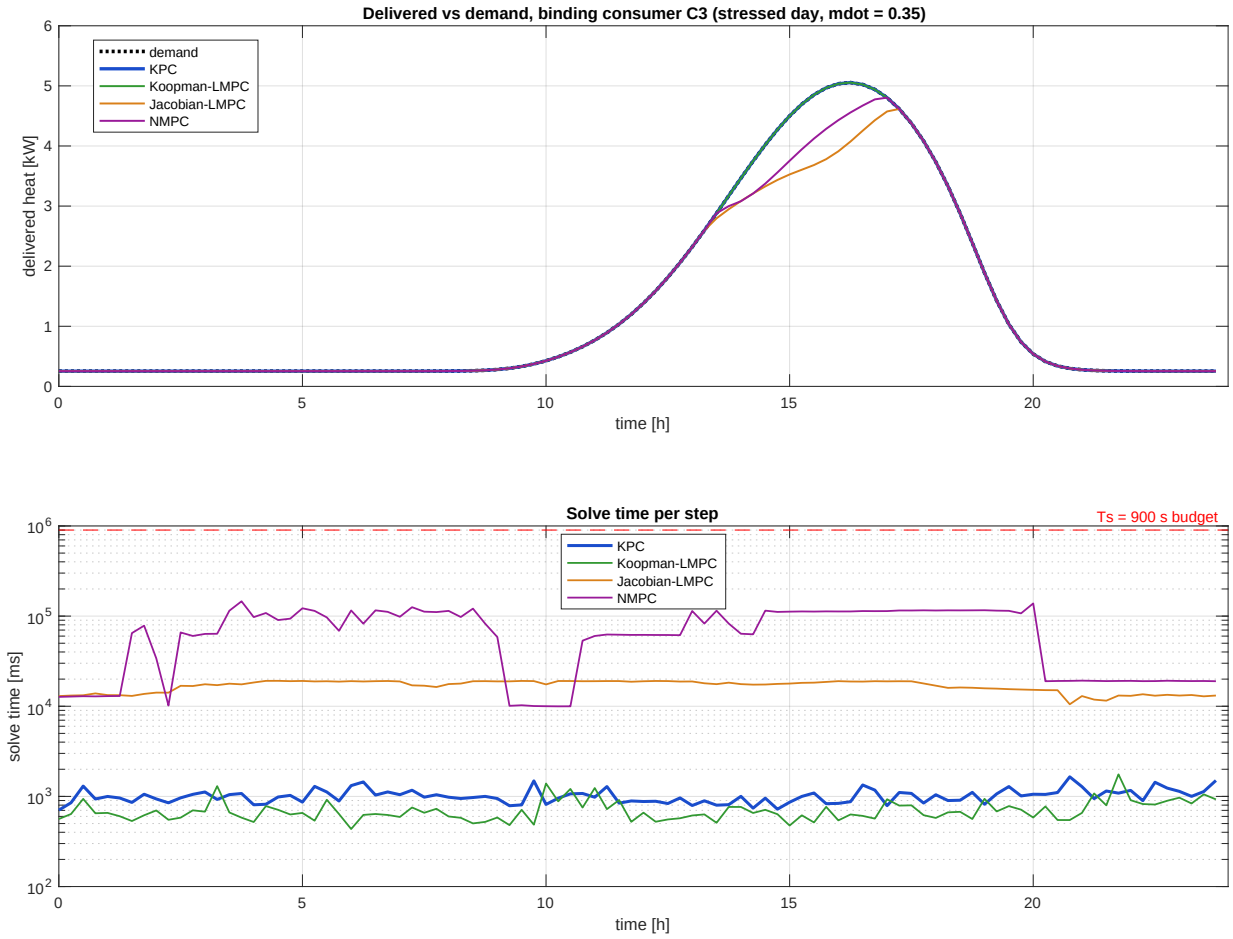

Figure 1: Closed-loop tracking at C3, the consumer that binds *across the four controllers*, and per-step solve times. C3 is where Jacobian-LMPC is worst (91.62 %); the two Koopman legs serve it fully, and their own headline number (97.56 %) is set at C5, which is not the panel drawn here. The solve-time panel is logarithmic; the red dashed line marks $T_s = 900$ s, above which a solve is too slow for real time.

The two Koopman controllers hold delivered heat on demand except inside the capacity-limited window, where the network physically cannot serve the worst consumer and a correct controller rides the constraint set. The Jacobian-LMPC under-delivers because one linearisation per step misjudges the bilinear flow-temperature coupling that the learned lift carries in its features. The order-of-magnitude dips in the NMPC solve-time trace are the steps where `fmincon` reaches a first-order optimum quickly (about 19 s median against about 111 s for stalled steps).

3.2 Per-step solver certificate (`viz_airtight_optimality.pdf`)

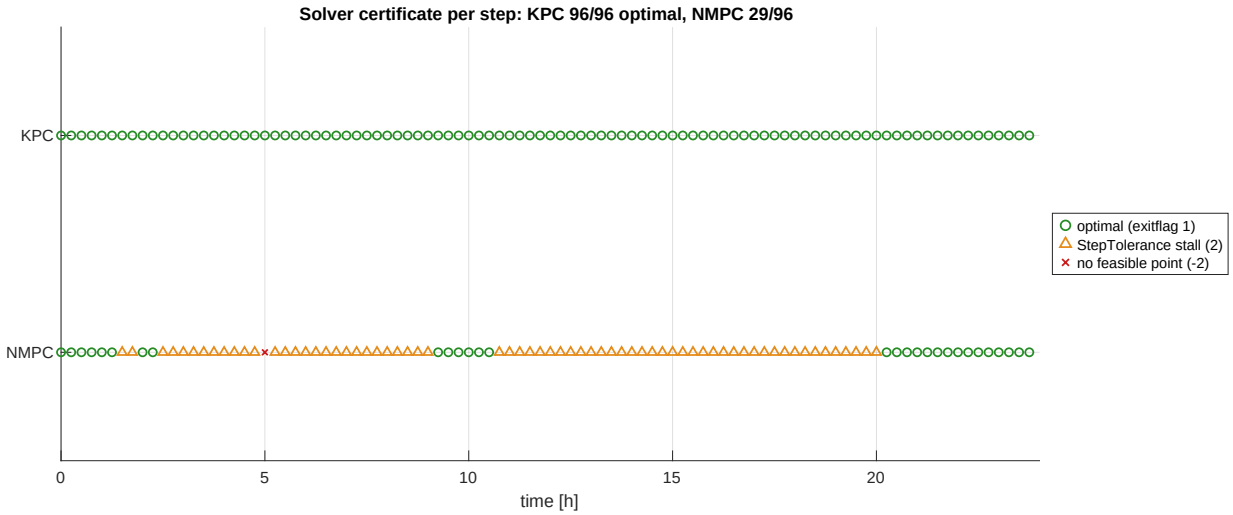

Figure 2: Per-step solver certificate from the logged exitflags: one marker per control step. KPC is 96 certified optima; NMPC reaches first-order optimality on 29 steps, stalls at `StepTolerance` on 66 (orange), finds no feasible point once.

The Jacobian-LMPC row is not drawn (identical to KPC, 96 certified optima); the Koopman-LMPC would show 93 optima and 3 tolerance stalls. The contrast is expected: a convex QP with soft slacks is feasible by construction, while the exact-plant rollout makes NMPC nonconvex, so `fmincon` legitimately stalls on flat or rough regions.

3.3 Shared-horizon fairness (`viz_airtight_npsweep.pdf`)

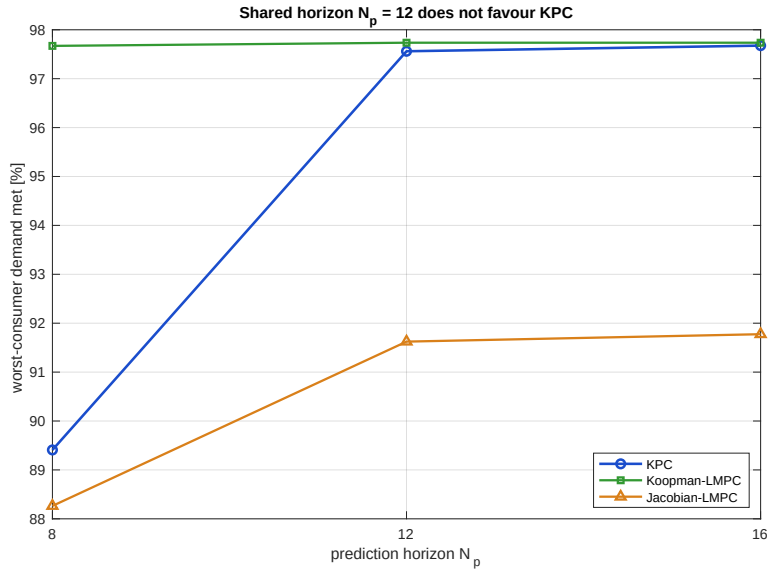

Figure 3: Shared-horizon fairness sweep, worst-consumer demand met against $N_p \in \{8, 12, 16\}$. The shared $N_p = 12$ sits at the knee where all three have converged; at $N_p = 8$ the iterated baseline is clearly better, so sharing KPC's tuned value biases against KPC, not for it.

NMPC is omitted from the sweep because it dominates the roughly three-hour run time at every horizon.

4 The automated checks

Three layers back the table, each catching a different failure.

Setup-fidelity gate. `sanity_airtight.m` (lines 59–61) re-runs the two Koopman legs and asserts $|\text{worst} - 97.561| < 0.05$ and $|\text{worst} - 97.735| < 0.05$, so the shared setup is production, not a diverged copy. It also asserts polytope feasibility (`linprog`, lines 79–80, 94) and that both baseline solvers converge and reduce the true objective. The three-hour run never starts on a silently broken baseline.

Independent pin in the test suite. `tests/validate_robustness.m` asserts the same two comfort numbers on the production `comparison.mat` (lines 73–76) and a KPC-versus-NMPC speed ratio of at least 10 (lines 61–62). The comfort headline is pinned in two independent places. The 81.0 ratio (80 346 / 992 ms, both medians from the same run) is machine-relative, so the test carries a machine-independent floor instead.

Honest solver accounting. Exitflags and iteration counts are logged per step from the `fmincon` output and tallied at the end, including a cap-hit count. Because the NMPC budget is demonstrably slack (max 4 of 1000 iterations), its non-optimality on 67 of 96 steps is a property of the nonconvex problem, not a starved solver.